

# Bases de Datos 2

Teoría 08

Julián Grigera

## MongoDB

Escalabilidad



### Distribución

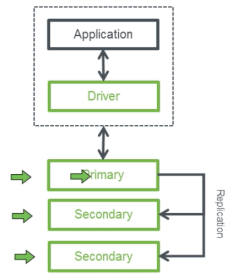
Replicación y *Sharding*



#### Ventajas y beneficios del modelo de distribución

- La carga se distribuye horizontalmente en todo el cluster.
- Capacidad de almacenamiento
- Si el espacio requerido aumenta, simplemente se agregan nuevos nodos al cluster.
- Alta disponibilidad
- Aún cuando algún nodo no funcione, se puede seguir resolviendo parcialmente el pedido.

## Replica Sets MongoDB



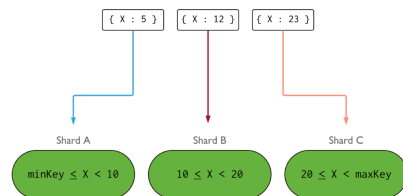
Un RS es un conjunto de nodos de MongoDB que comparten la misma información.

Los RS también mantienen la consistencia. Se puede hacer mantenimiento sin perder disponibilidad.

Configuración: se pone uno como primario preferentemente (después se administra solo).

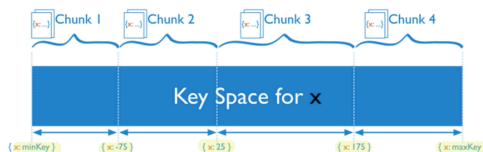
Cuando el nodo primario se cae, los nodos de recovery negocian cuál será el nuevo primario. Cuando vuelve, se pone él mismo como recovery. Por esto, todos los nodos deben tener la capacidad de ser nodo primario.

## Sharding MongoDB



- Mongo realiza la partición a nivel de colecciones utilizando una “shard key”.
- Está compuesta de uno o más campos inmutables.
- Solamente puede existir una shard key por colección.
- Si cada Shard correspondiera a un server, se podría complicar el escalado
  - Por esto, se busca una representación intermedia más flexible
  - Con la *shard key* se arman **chunks** - algo similar a percentiles del espacio de valores de la ShK

## Sharding MongoDB



Chunks y Shards NO es 1 a 1. Pueden haber varios Chunks en un solo Shard. Se intenta distribuir los Chunks **homogéneamente**. Los Chunks se pueden mover de Shard a Shard.

Elegir una clave de sharding es importante (y difícil) porque determina la búsqueda de las queries. Una buena clave de sharding:

- tiene muchos valores diferentes posibles. Ej. “país” de una persona es un mal chunk - la ciudad sería mejor.
- distribuye de forma pareja. El ejemplo de la ciudad no es bueno en este caso. Hay ciudades muy grandes y otras muy chicas. El chunk de NY va a crecer

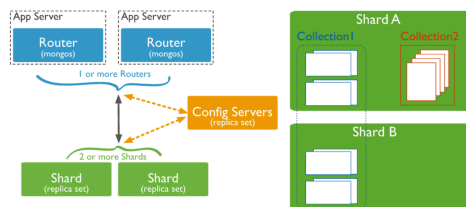
demasiado. Buen ejemplo: nro de teléfono.

- no debe crecer de forma monotónica. Ejemplo: el Timestamp de nacimiento (alta cardinalidad, buena distribución). Si empieza a nacer gente nueva, todas las inserciones van a ir al mismo Chunk, y por ende al mismo Shard/Server. Lo vamos a matar, el chunk se parte y va a otro shard, pero el último chunk sigue quedando en el mismo.

La más importante es que esta clave sea usada en las queries. Se pueden crear claves combinadas, ejemplo ciudad de nacimiento y nro de teléfono.

## Distribución en MongoDB

### Shards y Replica Sets



- Cada “shard” contiene un subconjunto de la información partida.
- A cada parte se la distribuye como “replica set”.
- Mongos: actúan como los puntos de entrada de los clientes para *rutear* las consultas.
- Config servers: almacenan la configuración y metadata relacionada con el cluster.

## Sharding

Ejemplo completo

## Ejemplo

Banco - Tarjetas de Crédito

```
{
  "nombreTitular": "María López",
  "cuit": 20123456783,
  "numero": "4024007198753210",
  "vencimiento": "2028-07",
  "limiteCredito": 6000000
}
```

Esto crea una colección shardeada con clave { cuit: 1 }

→ Range-based: cada chunk cubre un rango de CUIT (por ejemplo [20000000000, 20050000000)).

## Shard Keys

### Range (default)

Se divide por rangos reales de clienteld

→ Queries por rango

→ Riesgo de hotspot -> inserts van siempre al extremo superior

```
sh.shardCollection("banca.tc", { clientId: 1 })
```

### Hashed

Mongo **hashea** los valores - divide por ese valor

→ muy **equilibrado** para inserciones monótonas.

→ no sirve **para rangos (\$gt, \$lt)** porque el orden se pierde.

```
sh.shardCollection("banca.tc", { clientId: "hashed" })
```

Cuando se defina la colección shardeda, hay 2 tipos de shard key:

Range (por defecto) →

```
sh.shardCollection("banca.tc", { clientId: 1 })
```

Los chunks se dividen por rangos reales de clienteld.

→ útil para queries por rango, demo de splitAt, etc.

→ riesgo de hotspot si los inserts van siempre al extremo superior.

Hashed →

```
sh.shardCollection("banca.tc", { clientId: "hashed" })
```

Mongo *hashea* los valores y distribuye los chunks por ese hash.

→ muy equilibrado para inserciones monótonas.

→ no sirve para rangos (\$gt, \$lt) porque el orden se pierde.

```
{
  "nombreTitular": "María López",
  "cuit": 20123456783,
  "numero": "4024007198753210",
  "vencimiento": "2028-07",
  "limiteCredito": 6000000
}
```

```
use banco
sh.enableSharding("banco")
sh.shardCollection("banco.tc", { cuit: 1 })
```

Esto crea una colección shardeada con clave { cuit: 1 }  
→ Range-based: cada chunk cubre un rango de CUIT (por ejemplo [20000000000, 20050000000)).

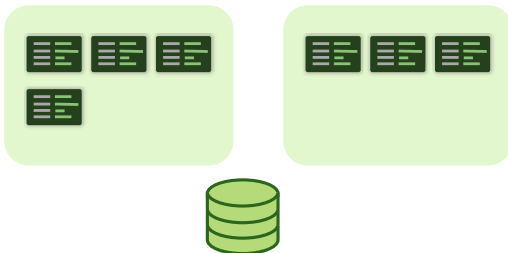
[20000000000 , 20999999999)



Primero hay solo un chunk que cubre todo el rango

[20000000000 , 20499999999)

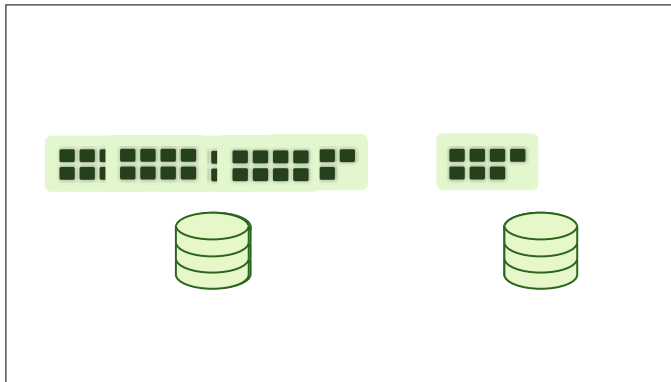
[20499999999 , 20999999999)



Cuando se supera el tamaño de un chunk,

- Mongo elige el split point automáticamente cuando un chunk supera el tamaño máximo (por defecto 64 MB).
- Divide el rango en dos mitades según la distribución de la shard key observada (en este caso, los cuit).
- Toma una clave media aproximada dentro del chunk como punto de corte.
- Así mantiene los chunks de tamaño similar y balanceados.

**Atención:** quedan ambos chunks en el mismo shard *en principio*



## Entra el **balancer**

Es un proceso en background (dentro del mongos) que:

- se ejecuta periódicamente,
- detecta si algún shard tiene más chunks que otros,
- y **migra chunks en segundo plano** hasta equilibrar la carga.

Funciona muy parecido a un **garbage collector**: automático, silencioso, pero puede pausarse o incluso forzarse

## Sharding

### Consideraciones

- Se puede elegir modo “**hashed**”
  - convierte los valores elegidos a un hash
  - no sirve para consultas por > o <
- Las **otras** colecciones, no-shardeadas
  - Siguen viviendo cada una en su propio shard
  - No se hace replicación (entera, en este caso) **hacia otros replica sets**
- Todo se puede hacer manualmente:
  - partir chunks eligiendo el punto medio
  - migrar chunks de un shard a otro

Las colecciones *no shardeadas* viven **completas en un solo shard** (en su replica set).

Mongo las llama **unsharded collections**, y para cada base shardeada elige un “primary shard” donde se guardan.

## NoSQL

### Consistencia

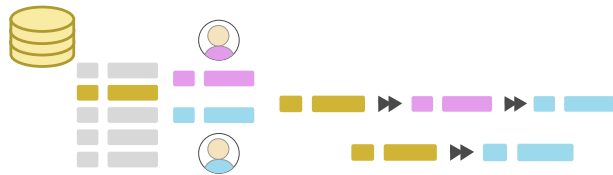
En los modelos de distribución, la consistencia agrega una complejidad

En RDBMS / monolito clásico, el escenario es distribuido, pero la BD es una sola

En NoSQL con distribución horizontal, se abren nuevos escenarios

- Aparecen los compromisos entre consistencia y performance

## Consistencia RDBMS

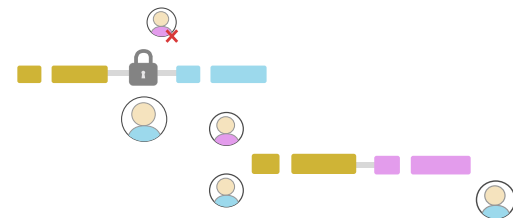


Las BBDD relacionales buscan la consistencia a todo costo

Ejemplo de escritura simultánea

- Hay una edición que se pierde: la que se aplica finalmente debía sobrescribir el estado original (amarillo), sin embargo termina sobrescribiendo el intermedio (rosa).
- Para evitar esto, hay dos soluciones clásicas...

## Inconsistencias de Escritura Manejo en RDBMS



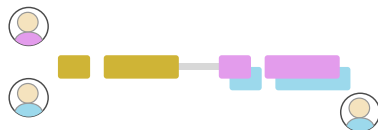
1) Locking pesimista: previene el conflicto

En el ejemplo, se serializan las ediciones pero la primera obtiene el lock e impide la segunda

2) Locking optimista: deja ocurrir el conflicto, pero actúa en consecuencia

Un ejemplo sería actualización condicional, donde se verifica que el valor no haya cambiado desde la última lectura antes de actualizar.

## Inconsistencias de Escritura Manejo en RDBMS



Otro sería el uso de merge, tal como se hace en un repositorio de código.

En un solo servidor, estos casos tienen sentido.

En un cluster, hay otras opciones. Se habla de consistencia secuencial.

Siempre hay un trade-off entre seguridad y velocidad de respuesta

## Inconsistencias de Lectura

Manejo en RDBMS



- Dos entidades deben mantener la consistencia
  - Actualización en etapas podría dejar una ventana inconsistente
- Para evitar este escenario clásico se utilizan las **tx**
- Las BBDD NoSQL (excepto grafo) tienen **tx** a nivel de **agregación**

## Consistencia y Distribución

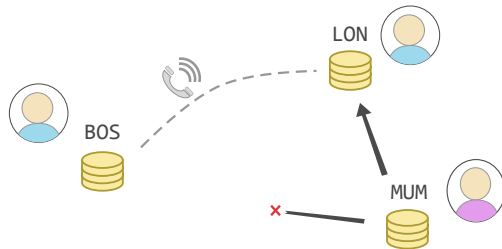
Escenarios en NoSQL



- Además, pensemos en escenarios distribuidos
  - Las BBDD NoSQL orientadas a agregaciones están preparadas para esto
- Replication + Sharding
- Master & Slave / P2P
- ¿Cómo pensamos la consistencia?

## Consistencia y Distribución

Escenarios en NoSQL



- Ejemplo, una habitación de hotel.
- Dos personas conversando por teléfono para tomarla, una en Londres y otra en Boston.
- Un 3ro la toma desde Bombai...
- Al mismo tiempo, la persona de Londres la ve libre y la de Boston tomada > las réplicas geográfica pueden tardar en actualizarse



## Consistencia y Distribución

### Escenarios en NoSQL

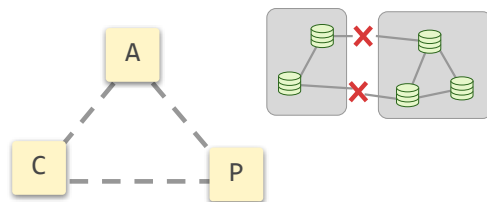


Ejemplo 2, un post.

- Un usuario escribe un comentario.
- Cuando lo guarda y la página se recarga, no lo ve > al haber balance de carga, al usuario le tocaron dos nodos diferentes: uno en la escritura y otro en la lectura - desactualizado
- Se puede buscar read-your-writes consistency con sticky sessions
- Se suele relajar la consistencia, incluso en RDBMS

## Teorema CAP

### Consistencia Disponibilidad (tolerancia a) Particiones

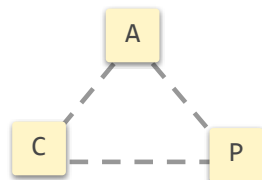


Tomado de “NoSQL Distilled”:

- “Given the three properties of Consistency, Availability, and Partition tolerance, you can only get two.”
- *Availability: if you can talk to a node in the cluster, it can read and write data.*
- *Partition tolerance: the cluster can survive communication breakages that separate the cluster into multiple partitions unable to communicate with each other*

## Teorema CAP

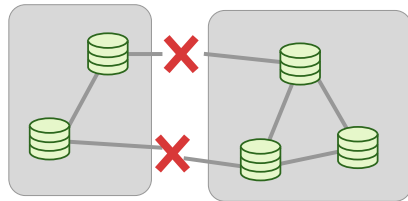
### Ejemplo



- Un solo server puede ser CA - dado que no se va a partir. RDMS son generalmente así.
- Se podría generar un cluster CA - si ocurre una partición, se apagan todos. De esta forma “Availability” según CAP se cumple igual.
- Todo radica en la P: cuando las particiones ocurren, hay que sopesar C y A

## Teorema CAP

### Ejemplo



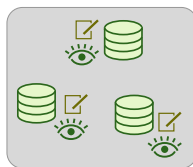
- Dos personas quieren una habitación. Si se reserva en el nodo de San Pablo, éste se comunica con el de Buenos Aires antes de confirmar > si se desconectan, se pierde la disponibilidad
- Se puede designar un nodo master para el hotel. Si es SP, entonces los usuarios de SP tienen disponibilidad. Los BA pueden ver la inconsistencia pero no reservar.
- Se puede permitir que BA (slave) también acepte reservas aunque se desconecten > overbooking
- Ejemplo de shopping cart en Amazon > se utiliza un merge al reconectar

## Quorum

### Consistencia



$$W > N/2$$



$$R + W > N$$

¿Cuántos nodos necesito para obtener consistencia? Si hay un master, uno. Si es P2P

Escritura: no hace falta que todos admitan una escritura, sólo la mayoría.

Lectura: se consultan nodos hasta confirmar la mayoría

Se puede formalizar como:  $R + W > N$

## En MongoDB

### Ejemplo de Quorums y estrategias

| Tipo                 | Descripción                                                                                   | Usos                                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| local (default)      | Lee lo que tenga el nodo local, sin esperar replicación ni consenso.                          | Lecturas rápidas donde toleras ver datos ligeramente viejos o no confirmados. Logs, métricas, dashboards internos. |
| available (sharding) | Devuelve lo que esté disponible, incluso si falta un shard o hay inconsistencia entre shards. | E-commerce global, catálogos grandes, vistas "best-effort" donde es mejor mostrar algo que fallar.                 |
| majority             | Solo ve datos que alcanzaron a la mayoría de los nodos.                                       | Reservas, stock, estados de cuenta, cualquier cosa donde leer datos viejos rompa la lógica de negocio.             |
| snapshot             | Foto consistente de los datos dentro de una transacción.                                      | Reportes, análisis, operaciones largas que necesitan una vista estable de los datos.                               |

# En MongoDB

## Ejemplo de Quorums y estrategias

| writeConcern                                    | Qué significa                                                            | Cuándo usarlo                                                                |
|-------------------------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------|
| { w: 0 }                                        | El cliente no espera confirmación de nada (ni primario).                 | Perder datos no importa (inusual).                                           |
| { w: 1 }                                        | Se considera exitosa cuando la recibe el primario (no espera réplica).   | Alta escritura, se tolera perder las últimas operaciones si el primario cae. |
| { w: N }                                        | Espera confirmación de N nodos (incluido el primario).                   | Casos muy específicos.                                                       |
| { w: "majority" }                               | Espera confirmación de la mayoría de los nodos del replicaset.           | Casos críticos: reservas, dinero, estados.                                   |
| { w: 1, j: true }<br>{ w: "majority", j: true } | Además, asegura que la operación se haya escrito en el journal del nodo. | Cuando la durabilidad frente a crash de máquina es importante.               |

# En MongoDB

## Ejemplo de Quorums y estrategias

```
db.collection("users").withWriteConcern({
  w: "majority",
  wtimeout: 5000
});
```

```
users.findOne(
  { name: "Ana" },
  { readConcern: { level: "local" } }
);
```

Lectura rápida, sin esperar consenso

Escritura rápida, solo primario

```
users.withWriteConcern({ w: 1 }).insertOne({
  name: "EventoLog",
  ts: new Date()
});
```